CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
ENGENHARIA ELÉTRICA

CHRISTOPHER CÉSAR DE MELO ANDRADE

# LABORATÓRIO DE SISTEMAS MICROPROCESSADOS
# RELATÓRIO: PRÁTICA 7

Belo Horizonte, 13 de Novembro de 2025

Para todos os programas apresentados neste relatório, as portas A, D e E foram configuradas como saída, permitindo o controle direto dos displays de 7 segmentos. Além disso, o registrador INTCON foi configurado como 0xE0, habilitando as interrupções globais, periféricas e especificamente a interrupção por overflow do Timer0. O registrador T0CON foi definido como 0x86, o que ativa o Timer0 em modo de 16 bits, seleciona o clock interno, define a contagem na borda de subida e ajusta o prescaler para 1:128, garantindo uma temporização adequada para a aplicação.

Com essas configurações de setup, é possível definir um valor inicial para o TIMER0 de forma a atingir uma interrupção por overflow do timer de 1KHz (1ms).

$$F_{tmrclk} = \frac{\frac{20 * 10^6}{4}}{128} = 39.063$$

$$TMR0 = 65536 - \frac{39.063}{1000\ Hz} = 65497$$

Assim, o Timer0 deve iniciar em 65497, garantindo que a cada 1 milisegundo uma interrupção seja gerada, servindo de base de tempo para os três programas.

1.

```c
/*
 * File:   ex01.c
 * Author: Christopher Andrade
 *
 * Created on 13 de Novembro de 2025, 12:01
 */

#include <xc.h>

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config MCLRE = ON
#pragma config PWRT = ON
#pragma config XINST = OFF

#define ON   (0x01)
#define OFF  (0x00)

int count = 0;
char clock = 0;

char display7seg[10] = { 0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x27, 0x7F, 0x6F };
```

```c
void interrupt timer()
{
    if(INTCONbits.TMR0IF)
    {
        INTCONbits.TMR0IF = 0;

        count += 1;

        if(count >= 1000)
        {
            count = 0;
            clock += 1;

            if(clock >= 10)
                clock = 0;
        }

        LATD = display7seg[clock];
        TMR0 = 65497;
    }
}

void main(void)
{
    ADCON1          = 0x0F;
    TRISA           = 0x00;
    TRISE           = 0x00;
    TRISD           = 0x00;
    INTCON          = 0b11100000;
    T0CON           = 0x86;
    TMR0            = 65497;
    PORTEbits.RE0   = ON;
    PORTEbits.RE2   = OFF;

    while(1){}

    return;
}
```

No primeiro programa, como toda a PORTA E foi configurada como saída, o bit **RE2** é inicializado para garantir que apenas o display esteja ativo no início da execução. A atualização do valor exibido ocorre dentro da rotina de interrupção do Timer0, onde a variável ***count*** registra quantas interrupções já ocorreram. Quando *count* atinge o valor 1000, correspondendo aproximadamente a 1 segundo, ela é zerada e a variável ***clock***, responsável por indicar o dígito mostrado no display, é incrementada. Como o vetor ***display7seg*** possui apenas dez posições, contendo os padrões para os números de 0 a 9, o código limita o valor de *clock* entre 0 e 9, reiniciando a contagem ao ultrapassar o último índice. O resultado é uma contagem crescente de 0 a 9, retornando automaticamente a zero após o dígito 9, com atualização do display a cada segundo.

2.

```c
/*
 * File:   ex02.c
 * Author: Christopher Andrade
 *
 * Created on 13 de Novembro de 2025, 12:01
 */

#include <xc.h>
```

```c
#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config MCLRE = ON
#pragma config PWRT = ON
#pragma config XINST = OFF

#define ON   (0x01)
#define OFF  (0x00)

int count = 0;

char clock = 0;

char display7seg[10] = { 0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x27, 0x7F, 0x6F };

int GetUnidade(char num) return (int) num % 10;

int GetDezena(char num) return (int) num / 10;

void interrupt timer()
{
    if(INTCONbits.TMR0IF)
    {
        INTCONbits.TMR0IF = 0;

        if(PORTEbits.RE0)
        {
            LATD = display7seg[GetUnidade(clock)];
            PORTEbits.RE0 = 0;
            PORTEbits.RE2 = 1;
        }

        else
        {
            LATD = display7seg[GetDezena(clock)];
            PORTEbits.RE0 = 1;
            PORTEbits.RE2 = 0;
        }

        count += 1;

        if(count >= 1000)
        {
            count = 0;
            clock += 1;
            if(clock >= 60)
                clock = 0;
        }

        TMR0 = 65497;
    }
}



void main(void)
{
    ADCON1          = 0x0F;
    TRISA           = 0x00;
    TRISE           = 0x00;
    TRISD           = 0x00;
    INTCON          = 0b11100000;
    T0CON           = 0x86;
    TMR0            = 65497;
    PORTEbits.RE0   = ON;
    PORTEbits.RE2   = ON;
```

```
    while(1){}

    return;
}
```

Neste programa, mantém-se a lógica básica do ex01.c, a variável ***count*** contabiliza as interrupções do Timer0 e a variável ***clock*** armazena o valor corrente do cronômetro. Entretanto, como agora é necessário exibir valores no intervalo [00, 59], dois displays de 7 segmentos são utilizados simultaneamente. Para isso, os bits **RE0** e **RE2** da PORTA E são configurados como saídas e usados para habilitar alternadamente cada display, permitindo a exibição de dois dígitos utilizando os mesmo barramento, técnica conhecida como multiplexação**.**

Para separar o número armazenado em ***clock*** em dois valores de dezena e unidade, foram criadas as funções ***GetUnidade*** e ***GetDezena***, ambas retornando valores no intervalo [0, 9].

A função ***GetUnidade*** utiliza a operação de módulo (*num % 10*) para extrair apenas o último dígito, enquanto ***GetDezena*** divide o número por 10 e descarta a parte fracionária, isolando a dezena. Esses valores são então usados como índices no vetor **display7seg**, que contém os padrões binários que formam cada número no display.

O processo de multiplexação ocorre dentro da rotina de interrupção: a cada interrupção, o código verifica qual display está ativo e alterna entre eles. Quando **RE0 está ligado**, exibe-se a **unidade**; quando **RE2 está ligado**, exibe-se a **dezena**. Essa comutação é feita em alta velocidade, muito mais rápido que a percepção humana, fazendo com que os dois dígitos pareçam acesos simultaneamente. Ao final de cada ciclo de 1000 interrupções (aproximadamente 1 segundo), a variável *clock* é incrementada e limitada ao intervalo [0, 59], garantindo um cronômetro que reinicia ao atingir 60 segundos.

3.

```
/*
 * File:   ex03.c
 * Author: Christopher Andrade
 *
 * Created on 13 de Novembro de 2025, 12:01
 */

#include <xc.h>

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config MCLRE = ON
#pragma config PWRT = ON
#pragma config XINST = OFF

#define ON   (0x01)
```

```c
#define OFF  (0x00)

int count = 0;
char clock = 0;

char display7seg[10] = { 0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x27, 0x7F, 0x6F };

int GetMinutes(char num)    return (int)(num / 60);

int GetSeconds(char num)    return (int)(num % 60) % 60;

int GetUnidade(char num)    return (int) num % 10;

int GetDezena(char num)     return (int) num / 10;

void interrupt timer()
{
    if(INTCONbits.TMR0IF)
    {
        INTCONbits.TMR0IF = 0;
        if(PORTAbits.RA5)
        {
            LATD = display7seg[GetUnidade(GetSeconds(clock))];
            LATEbits.LATE2  = 1;
            LATAbits.LA5    = 0;
        }

        else if(PORTEbits.RE2)
        {
            LATD = display7seg[GetDezena(GetSeconds(clock))];
            LATEbits.LATE0  = 1;
            LATEbits.LATE2  = 0;
        }

        else if(PORTEbits.RE0)
        {

            LATD = display7seg[GetUnidade(GetMinutes(clock))];
            LATAbits.LA2    = 1;
            LATEbits.LATE0  = 0;
        }

        else
        {
            LATD = display7seg[GetDezena(GetMinutes(clock))];
            LATAbits.LA5    = 1;
            LATAbits.LA2    = 0;
        }

        count += 1;

        if(count >= 1000)
        {
            count = 0;
            clock += 1;
            if(clock >= 3600)
                clock = 0;
        }

        TMR0 = 65497;
    }
}

void main(void)
{
    ADCON1          = 0x0F;
    TRISA           = 0x00;
    TRISE           = 0x00;
    TRISD           = 0x00;
```

```
    INTCON          = 0xE0;
    T0CON           = 0x86;
    TMR0            = 65497;
    PORTEbits.RE0   = ON;
    PORTEbits.RE2   = ON;
    PORTAbits.RA2   = ON;
    PORTAbits.RA5   = ON;

    while(1){}
    return;
}
```

Nesse programa, a lógica do cronômetro é estendida para quatro displays de 7 segmentos, permitindo contar até 1 hora completa (0 a 3600 segundos). Assim como nos exercícios anteriores, a variável *count* marca quantas interrupções ocorreram e ***clock*** é incrementada a cada segundo. Agora, porém, ***clock*** é limitada ao intervalo **[0, 3600)**, correspondendo ao total de segundos de uma hora. Para apresentar esse valor no display, é necessário primeiro converter o total de segundos em minutos e segundos, o que motivou a criação das funções ***GetMinutes*** e ***GetSeconds***. A primeira divide o valor total por 60 para obter os minutos decorridos, enquanto a segunda calcula o resto da divisão por 60, isolando os segundos. Em seguida, funções auxiliares ***GetUnidade*** e ***GetDezena*** extraem os dígitos individuais, permitindo exibir corretamente cada algarismo no vetor ***display7seg***.

O processo de multiplexação é ampliado para quatro dígitos: a cada interrupção, apenas **um** dos displays é ativado, alternando ciclicamente entre eles. O código verifica, em sequência, qual display está ativo (RA5, RE2, RE0 ou RA2) e atualiza o valor correspondente, seja unidade ou dezena de segundos, unidade ou dezena de minutos. Assim como nos programas anteriores, essa alternância ocorre em alta velocidade, rápida o suficiente para que o observador perceba todos os dígitos como acesos simultaneamente. Dessa forma, o programa exibe os minutos nos dois displays mais à esquerda e os segundos nos dois à direita, produzindo um cronômetro de quatro dígitos atualizado a cada segundo e reiniciado automaticamente após atingir uma hora.